WifiStepper UART host: initial draft

A direct Python 3 UART master for Good Robotics WSX100 controllers. Target: released stable firmware 002, with revision D hardware reported by the user. The physical revision-D header pin map has NOT been verified. This is not hardware-tested software and is not a safety-rated motion controller.

Deliberate scope

- One readable module, explicit `struct.pack` formats, synchronous requests.
- Existing firmware, existing UART daisy protocol; no firmware changes.
- No REST abstraction, server, workflow engine, background thread, UI, or scheduler.
- Only pyserial is needed for actual serial I/O; codec/tests use standard library.
- Operations are disabled unless `allow_control=True` is explicitly requested.
- No automatic stop, configuration change, clear-error, or command replay.

Source baseline and evidence limits

Release-associated source commit:
`3627d32739669bb249ec7a8760bd522925041a52`

Files inspected in goodrobotics/wifistepper:

- `firmware/wifistepper/daisy.cpp`
- `firmware/wifistepper/wifistepper.h`
- `firmware/wifistepper/powerstepO1.h`
- `firmware/wifistepper/command.cpp`
- `firmware/wifistepper/command.h`
- `documentation/src/_data/software.json`

Source: https://github.com/goodrobotics/wifistepper
Operation semantics: https://www.wifistepper.com/commands/motor.html
Serial API: https://pyserial.readthedocs.io/en/latest/pyserial_api.html

The release manifest associates version 2 with branch `stable` and image `wifistepper.image.2.bin`. The source identity is Wi-Fi Stepper/wsx100/stable/2. No reproducible binary-to-source verification has been performed.

Telemetry sizes are ABI predictions, not facts measured on the user's hardware. This draft requires CONFIG=228, STATE=87, SYNC=315 payload bytes and the identity above. Mismatches are errors, not an excuse to guess a different layout. The exact released-image enum/layout ABI remains a bench-validation item.

An important unresolved discrepancy: the public GoUntil/ReleaseSW queue-size tables and old Python client's one-byte action encoding do not establish that those ordinary C enums are four bytes in the released UART ABI. Consequently GoUntil and ReleaseSW are intentionally NOT implemented here. Their opcode constants are included for documentation only. Do not copy a guessed format. Similarly SETCONFIG, WIFI control, and queue persistence/copy APIs are deferred.

Install and offline tests

Python 3.10+ (tested here with Python 3.12):

```
python3 -m venv .venv
. .venv/bin/activate
python -m pip install -r requirements.txt
python -m unittest -v
```

The dependency file pins pyserial 3.5 for a reproducible starting point, not because it is asserted to be the newest available release.

Tests do not need pyserial. They exercise synthetic frames and a synthetic single-slave transport. They do NOT run the ESP8266 firmware, a cross-compiler, or actual hardware. Tests sharing assumptions with the implementation cannot prove those assumptions match the released binary.

Wiring prerequisites

Use a controller configured for daisy enabled, slave role; no other master. Logical topology for a single controller:

```
Pi TX  -> controller RX
Pi RX  <- controller TX
Pi GND -- controller GND
```

For multiple controllers route each TX into the next RX, with the last TX returning to the Pi RX. Do not parallel multiple TX outputs. Shared grounds are required by the documented topology. Verify revision-D connector pin numbers, orientation and logic voltage before connection. No power rail connection is specified here. Keep USB-programmer DTR/RTS disconnected from reset/boot pins; serial-port open can toggle modem-control lines. GPIO UART routing and console ownership must be checked locally. A PL011 UART is preferable to a mini UART. 512000 is a nonstandard baud rate: driver acceptance does not prove actual wire speed or adapter accuracy. Verify the actual waveform if necessary.

First use: observation only

After electrical verification:

```
python observe.py /dev/serial0 --passive --seconds 10 --log passive.jsonl
python observe.py /dev/serial0 --seconds 10 --log discovery.jsonl
```

The port path is an example, not a claim about the user's Pi mapping. Passive mode sends no protocol frames but still opens the serial device. Discovery mode sends PING and SYNC only. Heartbeats activate controller daisy bookkeeping and periodic telemetry, so this is observational, not literally state-free.
Log records contain raw byte chunks, including diagnostic text, with monotonic host timestamps. Chunks are not necessarily individual packets.

Equivalent API:

```
from wifistepper_uart import Host

with Host.open_serial('/dev/serial0') as host:
    print('Slaves:', host.discover())
    identity = host.identify(1)
    print(identity.model, identity.branch, identity.version)
    host.sleep(0.5)       # services receive and heartbeat
    print(host.state(1)) # raises if missing, stale, or incompatible
```

Single-thread ownership is required. Call poll() regularly or use host.sleep() while idle. Blocking requests also service receive/heartbeats. Do not perform long unrelated work or use time.sleep() for long delays while expecting the link to stay active. There is deliberately no hidden worker thread.

Basic operations

All methods return the command ID after firmware ACK. ACK is only handler receipt: firmware may acknowledge even when queue allocation failed. These methods do not wait for the physical motor operation to complete. Inspect fresh error/status telemetry. Existing faults are not cleared automatically.

Method	UART opcode	Payload
move(microsteps, direction=...)	24	`<BI`: packed direction, uint32 count
run(steps_per_second, direction=...)	22	`<Bf`: direction, float32 full steps/s
goto(position, direction=None)	25	`<BBi`: has-direction, direction, signed position
stop(hiz=False, soft=True)	21	two Boolean bytes; queued
immediate_stop(hiz=False, soft=False)	20	two Boolean bytes; bypass queue and empty Q0
step_clock(direction=...)	23	one direction byte; external STEP pulses required
go_home() / go_mark()	28 / 29	empty

Method	UART opcode	Payload
reset_position()	2A	empty
set_position(pos) / set_mark(pos)	2B / 2C	`<i`
wait_not_busy() / wait_stopped()	2E / 2F	empty; controller-side barriers
wait_ms(ms)	30	`<I`; UART payload is 4 bytes, internal queue uses 8
wait_switch(closed)	31	one Boolean byte
run_queue(stored_queue)	32	one byte
set_signal(value) / increment_signal(value)	33 / 34	`<i`
empty_queue()	40	empty; does NOT stop motor
clear_error()	05	empty; clears driver/firmware error indication

All opcode numbers in this table are hexadecimal. Most operations accept `target=1, queue=0`; queues 1..15 store commands rather than executing immediately. run_queue's destination is selected by `queue`, stored source by `stored_queue`. `clear_error` and `immediate_stop` are not queued.

Direction.FORWARD=1; Direction.REVERSE=0. Explicit direction enums are clearer than Boolean direction flags. If goto direction is omitted the driver chooses its normal shortest route; forcing a direction can cause position wraparound and a surprisingly long movement. Position is signed 22-bit microsteps; move count is unsigned 22-bit. No automatic splitting or wrapping is performed. Run speed is FULL steps/s independent of microstep resolution. A 200-full-step motor at 100 full steps/s is 30 RPM. move counts and positions ARE microsteps. The draft bounds speed to 0..15625 for wire/register sanity, NOT mechanical safety; actual permitted speed depends on configuration and mechanics.

GoHome means move to numeric position zero, not find a limit switch. wait_not_busy corresponds to legacy WAITBUSY (wait until BUSY deasserts). wait_stopped corresponds to legacy WAITRUNNING (wait until motor stops). After RUN, BUSY may deassert when the target speed is reached even though the shaft continues turning. A wait_stopped after endless RUN will block the queue; a later queued STOP behind that wait cannot execute. immediate_stop is the out-of-band firmware command, not a safety-rated physical emergency stop.

Defaults intentionally differ from the old library: stop keeps bridges active unless hiz=True, and immediate_stop defaults to hard stop with active bridges. Choose these explicitly for the mechanism. Neither default is universally safe. High impedance may release a loaded axis; active holding can cause heating.

Example bounded move and queue marker

Do NOT run this until the electrical mapping, firmware identity/layout, motor current settings, clearances and independent stop method have been verified. The count is illustrative, not a claim that it is safe for the mechanism. Keep REST and other writers idle while interpreting this sequence.

```
from wifistepper_uart import Host, Direction

with Host.open_serial('/dev/serial0', allow_control=True) as host:
    host.discover()
    host.identify(1)
    host.sleep(0.5)
    before = host.state(1)
    if before.errored or any(before.alarms[i] for i in (0, 1, 2, 3)):
        raise RuntimeError('Inspect controller fault before issuing motion')
    if not before.stopped:
        raise RuntimeError('Motor is not stopped')
    # Pick a marker distinct from the previous software signal.
    marker = before.signal + 1 if before.signal < 2147483647 else -2147483648
    host.move(16, direction=Direction.FORWARD)
    host.wait_stopped()          # enqueues a barrier; ACK is not completion
    host.set_signal(marker)     # executes after that barrier
    after = host.wait_signal(marker, timeout=10)
    print(after.position, after.busy, after.stopped)
```

The marker demonstrates progression through the execution queue, not exactly-once delivery, guaranteed insertion, mechanical accuracy, or settling. Because ACK does not guarantee queue insertion, a failed move can still be followed by a successfully inserted marker. Check errors and final position. Do not rely on last_command as a sole completion indicator: the source updates it through a pointer after queue-memory movement. Mechanical settling needs its own delay or measurement. wait_signal observes periodic STATE, not SYNC's zeroed signal.

Error and lifecycle policy

- No automatic command retries, reconnects, replay or error clearing.
- A request timeout or short write means outcome unknown, not command failed.
- Link timeout/topology changes invalidate cached identification and state.
- Same-count rearrangement is NOT reliably detected: re-identify after hardware changes; this draft is intended first for a fixed single-controller topology.
- A corrupt state/config frame disables the decoded profile until identification succeeds again; raw trace remains available.
- Identification gates operations, but does not prove the released binary ABI.
- Firmware heartbeat loss is NOT a motor watchdog; motion can continue.
- close()/context-manager exit sends no motor stop, even on an exception.

- A software stop can fail if the link/controller fails. Use an independent stop/power strategy appropriate to the mechanism. No hardware tested here.

Deferred on purpose

Motor-config serialization, full config-field decoding, GoUntil/ReleaseSW enum ABI resolution, queue persistence, automatic homing, trajectory planning, threading/asyncio, services, camera synchronization, fault recovery automation, and revision-D connector verification. The opcode enum documents additional wire names, not an assertion that every command has a high-level method.